SET 21

PART A — Short questions and answers

Q1. Explain how the multivalued attribute Session_Tags is mapped to a relational schema.

Answer: A multivalued attribute can have more than one value for one entity. A patient may have multiple phone numbers.

Q2. Describe the purpose of COMMIT and ROLLBACK after inserting registration records.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q3. Describe the redundancy caused by repeating Speaker_Name in every registration record.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q4. Compare UNION and UNION ALL.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q5. Summarize the order in which WHERE, GROUP BY and HAVING are applied.

Answer: GROUP BY collects rows with the same value so aggregate functions can calculate a result for each group.

Q6. Describe the purpose of a PL/SQL function that returns a value.

Answer: PL/SQL adds procedural programming to SQL. A procedure is a reusable named block that can take parameters.

Q7. Explain the trade-off between embedding and referencing data in documents.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q8. Describe the upsert option of an update operation.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

Q9. Describe the output of the MongoDB query: db.sessions.find({Tags: "AI"})

Answer: The query returns all documents that satisfy the specified filter condition.

Q10. Explain how CREATE adds a relationship between two matched nodes.

Answer: This concept helps organize, retrieve, or maintain database information. Use the example named in the question to explain it in one or two sentences.

PART B — System solution

Scenario: A technical conference maintains a Conference Management System to store attendee details, session

1(a) ER design:

• **ATTENDEE**(AttendeeID, AttendeeName, Organization, Email); primary key: **AttendeeID**.

• **SESSION_INFO**(SessionID, SessionTitle, SpeakerName, Room, SessionTime); primary key: **SessionID**.

• **REGISTRATION**(RegID, AttendeeID, SessionID, RegDate, TicketType); primary key: **RegID**.

Connect the two main entity tables through the transaction/relationship table. The transaction table has its own primary key and foreign keys to both entity tables. Cardinality: each entity can participate in many transactions; each transaction belongs to one row from each entity table (1:M on both sides, resolving M:N). Include the transaction's date/status/amount attributes shown in the question.

1(b) SQL DDL/DML: Create each table with its listed columns, PRIMARY KEY on its ID, and FOREIGN KEYs in the transaction table. Add NOT NULL to required fields; use CHECK/UNIQUE/DEFAULT where the question asks. Insert five sample rows per table, making sure foreign-key values exist.

```
CREATE TABLE ATTENDEE (
  AttendeeID NUMBER PRIMARY KEY,
  AttendeeName VARCHAR2(100),
  Organization VARCHAR2(100),
  Email VARCHAR2(100)
);
CREATE TABLE SESSION_INFO (
  SessionID NUMBER PRIMARY KEY,
  SessionTitle VARCHAR2(100),
```

```
  SpeakerName VARCHAR2(100),
  Room VARCHAR2(100),
  SessionTime VARCHAR2(100)
);
CREATE TABLE REGISTRATION (
  RegID NUMBER PRIMARY KEY,
  AttendeeID NUMBER,
  SessionID NUMBER,
  RegDate DATE,
  TicketType VARCHAR2(100)
);
```

2(a) Join query: Select the two entity names/details from the transaction table by joining the transaction table to both entity tables using their IDs.

```
SELECT a.*, b.*
FROM REGISTRATION x
JOIN ATTENDEE a ON x.AttendeeID = a.AttendeeID
JOIN SESSION_INFO b ON x.SessionID = b.SessionID;
```

2(b) Aggregate query: Group the transaction rows by the requested category/type and apply HAVING to the aggregate condition stated in the question.

```
SELECT grouping_column, COUNT(*) AS total
FROM transaction_table
GROUP BY grouping_column
HAVING COUNT(*) > threshold;
```

2(c) PL/SQL procedure: Create a procedure with an IN parameter for the requested ID. Join the transaction table to the related entity tables, filter by that parameter, and print selected values using DBMS_OUTPUT.PUT_LINE.

```
CREATE OR REPLACE PROCEDURE show_details(p_id IN NUMBER) AS
BEGIN
  FOR r IN (
    SELECT a.*, b.*, x.*
    FROM transaction_table x
    JOIN entity_table_1 a ON x.entity1_id = a.entity1_id
    JOIN entity_table_2 b ON x.entity2_id = b.entity2_id
    WHERE x.transaction_id = p_id
  ) LOOP
    DBMS_OUTPUT.PUT_LINE('Details retrieved');
  END LOOP;
END;
/
```

3. MongoDB: Create the collections and insert documents using the fields listed in the set. Query with find() and the requested filter.

```
db.collection.insertMany([
  { _id: 1, name: 'Sample A', category: 'Example', price: 500 },
  { _id: 2, name: 'Sample B', category: 'Example', price: 300 }
]);
db.collection.find({ author: 'Specified Author' });
db.collection.find({ price: { $gt: 400 } });
```